🧠 Engineering AI Operating System — Roles, Architecture & Technology Baseline

2026-10-02 Locked Product Clarification — Identity-on-Action + Merchant Credit Boundary

Identity-on-action

- Passive/read-only farmer activity does not require forced authentication.
- Authentication is requested at the first action requiring attributable identity, durable ownership/history, publication, communication, financial accountability or another authenticated write.
- Backend authorization remains authoritative.

Merchant credit MVP boundary — locked

- Shop Owner/Dealer is the commercial decision-maker/counterparty.
- Farmer and Shop Owner identities remain independent.
- Farmer requests credit under Shop Owner terms.
- Farmer explicitly reviews and consents to the displayed terms before submitting.
- Shop Owner may accept or issue one counteroffer.
- Farmer may accept/reject that counteroffer; no counter-counteroffer.
- Any changed counteroffer terms must be explicitly displayed and consented to.
- Shop Owner provides final approval before transaction commit.
- The Farmer App facilitates the workflow; it does not introduce a separate lending/underwriting/payment product in the MVP.
- Final legal wording is an operational/legal dependency, not a reason to reopen the product workflow.
- Future scaling and broader financial models remain separate architecture/business work.

Provider approval clarification

“Commercial approval” means only that external provider usage must have a valid commercial account/license and permitted access method before production. It is an implementation dependency, not an owner product question.

2026-10-02 Engineering Rule — Identity-on-Action + Flow Change Control

Identity-on-action

- Do not force authentication for passive/read-only Farmer App usage.
- Authenticate/gate only when a durable, attributable, public, financial, transactional, communication or otherwise identity-dependent action is attempted.
- Typical identity-gated actions: post, reply/comment, account-linked upload, dealer/Expert message, credit request/acceptance, field-visit request, persistent save/history and other accountable writes.
- The backend/business operation is authoritative for whether authentication is required; frontend gating alone is insufficient.
- Guest farming context should migrate safely to the authenticated account where permitted.

Flow simplification rule

An implementation agent may reduce a 3-step flow to 2, merge steps, reorder independent fields, or remove redundant UI **without changing the product contract**.

The agent must not silently remove a step if it is the only mechanism for required:

- data collection;
- terms/consent acceptance;
- identity/authorization;
- validation;
- state transition;
- audit record;
- duplicate protection;
- source-of-truth linkage.

If the same required information and controls are preserved elsewhere, the agent may simplify the UX.

Required impact analysis for flow changes

Before changing a flow:

Old flow → required information → business rules → state machine → permissions → data writes → source of truth → dependent screens/services → notifications → analytics → regression tests.

Classification:

- **UX-only change:** underlying contract identical → implementation can proceed within approved scope.
- **Behavior/data change:** contract changes → requires explicit product/technical approval before implementation.
- **Business/security/source-of-truth change:** never silently change; treat as a new decision/change request.

Recommendation/personalization protection

Changing the number/order of screens does **not** change agricultural knowledge, behavioral signals or recommendations by itself.

Those systems are affected only if the new flow changes:

- a confirmed farmer/crop fact;
- a behavioral signal definition/strength/source;
- a recommendation-triggering event;
- the data available to the recommendation engine;
- the canonical source of truth.

Therefore, a request such as “take this 3-step flow and make it 2 steps” is normally a presentation/UX optimization, not a change to the recommendation system.

2026-10-02 Research-Backed Engineering Baseline — Farmer App

Provider / domain findings

- Agricultural knowledge must use a curated evidence pipeline and approved knowledge store. Sources include Punjab Agriculture Department production technology/extension/pest-warning material, Pakistan DPP registered-pesticide information, CABI/PlantwisePlus, and FAO guidance. cite turn157665search3 turn157665search1 turn992107search0 turn157665search9 turn992107search9
- AMIS Punjab is the initial market-data source candidate. Punjab PITB describes AMIS as publishing agriculture commodity prices from 135 Punjab markets and historical reporting. Confirm permitted machine-readable access before production ingestion. cite turn545971search4
- Open-Meteo free API is non-commercial; do not use the free tier for the commercial Farmer App. Weatherbit is the current primary provider candidate because its paid plans explicitly include commercial-use licensing and an agricultural weather offering; keep the provider behind an adapter. cite turn992107search3 turn569431search2
- Text language coverage remains product-level; voice is capability-gated. Current Google speech documentation supports Urdu (Pakistan), Pashto, Punjabi and Sindhi variants, but Saraiki was not found in the supported Speech-to-Text list. Treat Saraiki voice as validation-dependent, not a guaranteed launch capability. cite turn306136search0 turn154729search3
- HEC's current degree-attestation system can provide stronger evidence for relevant expert credentials, but expert verification must also support manual issuer verification. cite turn465444search0
- PostGIS is the correct geospatial foundation for the 100 km dealer-radius requirement. cite turn720178search0
- Supabase Storage image transformations can reduce profile/community image payloads; video needs a separate processing path. cite turn561253search0
- Supabase Cron can run retention jobs; use automated deletion for raw media/technical traces according to approved policy. cite turn561253search7
- Supabase RLS + Auth remains the database-level authorization boundary; do not rely on frontend checks. cite turn741948search7 turn741948search8
- PostHog is a viable analytics candidate for React Native, product analytics and AI event instrumentation; sensitive AI/credit payloads should not be shipped to analytics or unrestricted session replay. cite turn312506search0 turn312506search9

Required implementation boundaries

1. External source adapters must isolate provider changes.
2. AI must consume approved knowledge/tools, not arbitrary internet answers.
3. Observed data, inferred signals, recommendations and business records remain separate objects.
4. Dealer inventory/product/transaction truth remains dealer-system owned.
5. Credit is a finite state machine with one counteroffer maximum and final Shop Owner approval.
6. Moderation and expert verification decisions remain human-audited Admin/Moderation actions.
7. Voice availability is determined by tested provider/language quality, not UI selection alone.
8. Raw media and sensitive technical payloads are not permanent history by default.
9. RLS/auth/business-layer authorization must be enforced server-side and verified independently.
10. All critical actions require idempotency/duplicate protection and auditable state transitions.

Suggested MVP operating stance

- broad farmer/crop onboarding;
- high-confidence agronomic recommendations only where evidence coverage exists;
- Urdu-first text + selected regional language text;
- voice capability enabled only per validated language/provider;
- Punjab-first market/weather/dealer operations;
- read-heavy offline support rather than full offline business operation;
- provider-neutral architecture from day one.

2026-10-02 Engineering Lock - Expert Verification, Moderation & Credit Workflow

Expert verification — locked requirements

- Model expert verification as an auditable verification record, not a boolean profile flag only.
- **Mandatory:** verified identity + at least one relevant, verifiable professional qualification/certification.
- Store evidence references and reviewer decision metadata for qualification/certification, experience, specialization and relevant platform history.
- Practical experience and community contribution are supporting evidence only; they do not substitute for the minimum credential requirement.
- Internal **Admin/Moderation** is the final approval/suspension/removal authority in the initial system.
- Design for certificate/re-verification expiry without coupling verification to community reputation.

Community moderation

- Report → moderation case → Admin/Moderation review → explicit action → audit record.
- AI may detect/prioritize risk; it must not independently make irreversible moderation decisions or award professional verification.

- Permanent deletion/removal requires an explicit moderation action.
- Moderation outcome notifications are policy-controlled.

Credit state machine

Farmer and Dealer/Shop Owner remain independent identities. Credit is an integration workflow only.

Primary state path:

Requested → Shop Owner Reviewing → [Accepted directly | Counteroffered] → Awaiting Farmer Decision → Awaiting Shop Owner Final Approval → Accepted/Rejected.

Rules:

- One credit request per underlying order/transaction context while an active request exists.
- Farmer must confirm the displayed terms/version before submitting the request.
- Shop Owner may create at most one counteroffer for that request.
- Counteroffer changes are explicit and auditable.
- Farmer may only accept or reject the counteroffer; no counter-counteroffer.
- After farmer acceptance, Shop Owner performs final acceptance/rejection.
- Only final Shop Owner acceptance commits the approved credit transaction.
- Terminal rejection must not create an approved credit transaction.
- Use idempotent/atomic business operations to prevent duplicate approval/commit.
- Preserve original request, counteroffer, both decision events, terms/version, actor, timestamps and resulting business transaction reference.

Do not implement a general negotiation engine unless a future product decision explicitly introduces one.

2026-10-02 Engineering Rule — User Module, Conversation Memory & Retention

Profile implementation

- Preserve the final Profile UI exactly; implementation must provide the data/state behind the locked design.
- Maintain separate concepts for:
  1. authenticated account identity,
  2. farmer preferences/settings,
  3. farming/crop context,
  4. dealer relationships,
  5. dealer-owned transaction/credit records.
- Guest profile state must not expose authenticated-only actions.
- Account deletion must be an authenticated controlled workflow and must not destroy dealer-owned business records directly.

- Profile location is the farmer's default context; temporary dealer-search location is separate.
- Language preference is explicit and persisted; location may suggest a regional language but must not silently override the user's choice.
- Profile settings such as notifications and language must be shared by the same centralized design/component system.

Conversation/history architecture

- Do not create an unbounded raw chat/media archive as the personalization mechanism.
- Separate:
  - user-visible conversation history,
  - structured farmer/crop context,
  - behavioral personalization signals,
  - raw media,
  - technical traces.
- Use structured signals for long-lived personalization. Raw artifacts are temporary.
- AI conversation sessions should retain a session identifier, intent/topic, associated crop where known, relevant context references, recommendation IDs, outcome/feedback signals and required audit metadata without requiring permanent retention of every raw audio/image payload.

Initial retention defaults — proposed

- Structured farmer context/crop history: account-lifetime while permitted by privacy policy.
- Text chat history: product-defined retention; storage size alone is not a reason to delete useful text history.
- Behavioral personalization signal weighting: strongest for approximately 90 days, followed by decay.
- Raw voice: 7 days.
- AI-analysis images/videos: 30 days after structured result, unless an active case requires longer.
- Technical traces/debug payloads: 14–30 days.
- Temporary/failed uploads: purge automatically.
- Exact durations are implementation defaults, not legal policy; final privacy/legal approval can change them.

Storage rule

Store the smallest durable representation that preserves product value.

Prefer the pattern: raw event → structured signal/summary → aggregate history, rather than permanently storing every raw artifact.

QA requirements

Retention must be tested with time-based fixtures:

- raw media expires automatically,

- structured signal remains as expected,
- personalization weight decays,
- expired media cannot be downloaded through stale URLs,
- account deletion applies the documented retention/deletion policy,
- no unauthorized user can access another farmer's conversation/context.

2026-10-02 Farmer App Product Sync — Locked Decisions for Engineering

Source: latest owner verification against the Farmer App PRD and final visual direction. This section is the engineering-facing interpretation of the accepted product decisions.

1. Final UI is the visual source of truth

- The final Farmer App screens in the linked PRD are locked visual evidence. Do not redesign them during implementation.
- Implementation must work backward from each screen: component → data → source of truth → logic → trigger → state → dependency → API/service → verification.
- A missing backend/data rule must be surfaced rather than invented from the visual design.

2. Guest-first onboarding

- Do not force email, phone, OTP, or account creation at app install/onboarding.
- The farmer can continue without an account and still use the core farming utility.
- Farming context may be collected before account creation.
- Guest and authenticated experiences use the same underlying farming-context, recommendation and personalization logic.
- When a guest later creates an account, useful locally stored farming context should be migrated/associated safely rather than re-entered unnecessarily.

3. Farmer identity and dealer identity are separate

- Farmer App account and dealer-side customer record are different systems/identities.
- Farmer signup must not query, match, merge or autofill from the dealer customer database.
- Dealer integration begins only when the farmer explicitly enters a dealer workflow.
- Dealer-owned business records remain dealer-system source of truth.
- Never infer a cross-system identity merely because phone numbers match.

4. Farmer context and multiple crops

- Acreage is not an onboarding eligibility requirement.
- Do not force Farm → Field hierarchy during MVP onboarding.
- Backend must support multiple active Crop Instances per farmer.

- Each crop instance may track cycle/season, start/planting information, estimated/current stage, activities, problems, inputs/treatments, harvest/outcome and signals.
- Confirmed facts and inferred signals must be stored distinctly.
- Estimated stage from timing is an inference unless confirmed.
- AI image interpretation is a candidate signal until appropriate confirmation.
- A farmer may explicitly select a primary/featured crop; system suggestions may use recent activity, repeated questions, recency and available importance evidence.

5. Agricultural AI boundary

- Customer AI is agriculture-domain only.
- Agriculture domain includes crop/farming questions, pests/diseases, cultivation, agricultural weather, crop/season planning, agricultural market/input questions and the farmer's own farming context/history.
- Off-domain topics such as politics, sports and unrelated general knowledge must not be answered from general model knowledge.
- AI may interact and explain, but canonical product, inventory, price, weather, dealer verification, transaction and credit facts must come from system sources.
- Use a knowledge-grounded pipeline: intent/domain gate → context extraction → approved knowledge/data → deterministic rules/tools where needed → AI explanation → confidence/safety check → structured response.

6. Agricultural knowledge and evidence system

- The engineering system must support an agricultural knowledge base with source, evidence, crop/problem/product relationships, version, review status and timestamps.
- Do not treat unrestricted internet/LLM knowledge as the agricultural source of truth.
- Product/treatment relationships must be maintained independently from dealer inventory IDs.
- Where regulated inputs are involved, store the relevant authoritative registration/label evidence when available and approved.

7. Recommendation / Insight as canonical object

- Create one canonical Recommendation/Insight record that can be consumed by Home, Notifications, crop detail, history and follow-up interactions.
- Do not calculate separate contradictory recommendation versions for different surfaces.
- Recommendation record must retain context, type/problem, priority, recommendation, evidence/inputs, source/knowledge version, confidence, validity window and lifecycle status.
- Recommendation lifecycle should support active, viewed, acted-on, dismissed and expired states where relevant.

8. Personalization / behavior signals

- Capture structured signals from search, AI conversation, scan topics, Community activity, product/dealer searches, viewed/saved/dismissed recommendations and confirmed context changes.

- Signals have source + timestamp and should decay in relevance over time.
- Behavior influences relevance/personalization but must not silently become a confirmed agronomic fact.
- Personalization optimizes farmer usefulness, not engagement/time-spent.
- Interest signals do not directly trigger notifications; they create candidate relevance that must pass importance, freshness and notification rules.

9. Weather and market data

- Weather facts must come from the configured provider. AI retrieves provider data for weather questions.
- Agricultural weather interpretation is a separate recommendation layer using weather + crop context.
- Market/mandi rates must come from approved market data sources and a normalized ingestion/adapter layer.
- Preserve historical observations; never overwrite history or invent missing rates.
- Market data records should retain commodity, market, observation time/date, unit, reported values, source and freshness/ingestion metadata.
- The same market service must power Market UI and AI market answers.

10. Dealer/product integration

- Dealer/shop owner is the same business role for the Farmer App integration boundary.
- Dealer inventory/business system remains the canonical source for product identity as exposed by the integration, variants, price, stock/availability, live/published status and dealer operational/verification status.
- Dealer can mark a product/listing as live for the Farmer App from the dealer inventory workflow.
- Farmer-side data is a synchronized read model, not a second inventory truth.
- No delivery/pickup capability is part of the current product.
- Matching path: agricultural problem/need → approved product knowledge → canonical product → live dealer listings → 100 km location/radius + relevance/availability → farmer result.

11. Dealer discovery

- Maximum search radius: 100 km.
- Default search location is the farmer's configured location; dealer search may use a different temporary search location without silently changing the default farmer location.
- Ranking uses relevance/availability, verified/operational state, proximity and transaction/response signals.
- Order volume is a signal, not the sole ranking criterion.
- Nearby must not be presented as equivalent to recommended unless the ranking logic supports that claim.

12. Languages and localization

- Default Farmer App/AI language: Urdu.

- Initial regional-language support: Punjabi, Saraiki, Sindhi and Pashto.
- Farmer can manually select language.
- Location may suggest a regional language but must never silently change the user's explicit language choice.
- AI text/voice should follow the selected language where speech quality is validated.
- Shared design-system components must support English LTR and Urdu/regional RTL where applicable without duplicating component logic.

13. Media handling

- Compress images and videos before upload where practical to reduce bandwidth/storage.
- Use adaptive compression rather than one fixed compression value.
- Normal community/profile media is optimized aggressively for mobile performance.
- AI diagnosis media keeps sufficient detail for visual analysis; a separate higher-quality temporary analysis representation may be generated.
- Raw media is temporary unless an explicit retention exception exists.

14. Farmer → Shop Owner field visit

- From a dealer/shop-owner profile, farmer can create a separate Field Visit / Crop Inspection Request.
- It is not normal chat and not credit.
- Initial lifecycle: Pending → Accepted / Rejected.
- Future extension may add Scheduled → In Progress → Completed / Cancelled.
- Later visit findings should become structured crop/issue history where appropriate rather than remaining only in chat.

15. Credit

- Dealer/shop owner decides eligibility.
- Farmer submits credit request.
- Dealer reviews and can accept or return modified commercial terms.
- Farmer accepts or rejects the returned terms.
- Duplicate concurrent requests for the same order must be prevented.
- Repayment/outstanding status is synchronized from dealer business records.
- The **Farmer↔Shop Owner credit workflow is finalized** at the product level: one request, at most one counteroffer, farmer accept/reject, then final Shop Owner accept/reject; only final Shop Owner acceptance commits the approved credit transaction. Remaining open work is the legal/commercial operating model, enforceability, penalties/fees policy and payment rails. Engineering must not invent those rules.

16. Analytics / AI session visibility

Separate the observability layers:

- Product analytics: farmer behavior and feature usage.
- AI analytics: intent, low-confidence cases, unanswered questions, recommendation outcomes and language usage.
- Technical observability: crashes, API/RPC failures, sync failures, latency and notification failures.
- Store useful structured signals longer-term; retain raw voice/images/technical traces only for a defined limited period.
- Do not enable unrestricted session replay over sensitive AI/credit content without a reviewed privacy design.

17. QA requirement for all of the above

Every implementation must be verified across:

Requirement → business rule → source of truth → backend → database → authorization → frontend → UI state → cross-screen consistency → E2E → regression → release.

- Home and Notification must be explicitly checked against the same Recommendation/Event ID.
- Dealer inventory shown to the farmer must be checked against dealer-system truth.
- Market/weather figures shown in UI must be traceable to source + timestamp.
- AI answers must be checked for domain gating, grounding, confidence/safety and factual-source boundaries.
- Media upload must be tested on representative low-bandwidth/mobile conditions.
- Guest → account migration must preserve approved farming context without creating duplicate records.

18. Development-agent architecture remains OPEN

- Do not implement or lock the development-agent/orchestrator worker architecture from assumption.
- Feasibility, concurrency, task handoff, execution isolation, checkpointing, cost, speed, failure/retry behavior and runtime limitations must be researched first.

2026-10-01 Architecture Update

Farmer App baseline decisions from the owner: Android + iOS mobile app, Supabase backend, OpenAI, GitHub, GitHub Actions. Technical provider details are to be researched and proposed by the architecture role.

This page is the persistent engineering/AI operating system for the Farmer App project. It consolidates the previously developed Agri Shop AI architecture, QA architecture, engineering-memory model, verification rules, technology baseline, and agent-role definitions so future AI agents do not repeatedly ask for the same context.

1. Authority and Status Model

Every item in this document is classified as one of:

- **LOCKED / ACCEPTED** — must be followed unless a new decision explicitly supersedes it.
- **CURRENT BASELINE** — observed in the existing Agri Shop implementation; do not assume it is automatically the Farmer App production stack.

- **PROPOSED** — recommended target architecture; requires approval before being treated as locked.
- **OPEN DECISION** — the agent must ask or surface it before implementation.
- **EVIDENCE REQUIRED** — behavior/claim cannot be considered verified without direct evidence.
- **SUPERSEDED** — historical only; never silently reuse.

The agent must never convert Proposed or Open Decision material into a requirement by assumption.

2. Core Engineering Principle

The development system follows:

Requirement → Context + Business Rules → Impact Analysis → Plan → Implementation → Independent Verification → Evidence → Regression → Human Release Approval → Project-State/Memory Update

The implementation agent and verification role are logically independent even when the same underlying AI/runtime performs both roles.

No agent may claim **VERIFIED**, **PASS**, **FIXED**, or **DONE** without evidence appropriate to the change.

3. Logical Agent / Role Architecture

A. Orchestrator — controller

Purpose: Own the overall task lifecycle and route work to the correct role/capability.

Responsibilities:

- Parse the request.
- Identify requirement intent.
- Retrieve PRD, decisions, business rules, architecture, design evidence, code context, and prior learnings.
- Check for conflicts with accepted decisions.
- Perform impact analysis.
- Select the required roles.
- Track implementation and verification status.
- Require evidence before accepting a result.
- Update project state and memory after completion.

The Orchestrator does not invent business rules.

B. Product Designer

Owns:

- User problem and intent.
- Journey and information architecture.
- Interaction model.
- Information hierarchy.

- Accessibility/localization/RTL.
- Design-system consistency.
- UX trade-offs.
- Design validation.

For every screen/component it asks:

Why → user intent → context → priority → interaction states → system data → second-order effects → trade-off.

C. Technical/Product Leader

Acts as the CTO/product-engineering role.

Owns:

- Architecture.
- Data ownership.
- Source of truth.
- State machines.
- Permissions/security.
- Failure modes.
- Idempotency/retry behavior.
- Audit/history.
- Migration/backward compatibility.
- Performance.
- Cost.
- Third-party dependencies.
- Maintainability and feasibility.

D. Investigator

Authority: read-only.

Inspects:

- Code.
- Schema.
- RPCs/functions.
- RLS/security policies.
- Logs.
- Existing tests.
- Runtime evidence.
- Existing decisions and learnings.

Output:

- Findings.
- Evidence.
- Confidence classification.
- Affected files/entities.
- Suspected root cause, clearly marked as hypothesis until verified.

E. Planner

Authority: propose only.

Produces:

- Ordered implementation plan.
- Affected UI/backend/data/test areas.
- Acceptance criteria.
- Required verification.
- Regression scope.
- Risks and dependencies.
- Human approval gates.

F. Implementer

Authority: autonomous only for Tier-1 safe/routine work.

Can:

- Implement approved changes.
- Write tests.
- Run tests.
- Update documentation with evidence.
- Fix routine bugs when regression coverage is included.

Cannot silently change:

- Business rules.
- Financial logic.
- Security architecture.
- Authentication model.
- Major schema behavior.
- Production data.

G. Lead QA Mike

Purpose: active, independent development QA.

Runs after implementation and challenges the implementer's claim.

Responsibilities:

- Reproduce the reported problem.
- Verify expected vs actual behavior.
- Inspect database state directly where relevant.
- Verify business rules.
- Verify permissions/security.
- Verify RPC/API behavior.
- Run E2E/browser checks.
- Run regression suites.
- Check cross-screen consistency.
- Reject unsupported “fixed” claims.
- Produce structured evidence.

Lead QA Mike must not accept frontend-only changes when the actual defect is backend/data/business-rule related.

H. Mike Assistant

Purpose: continuous/background QA and engineering surveillance after development/release.

Looks for:

- Regressions.
- Data drift.
- Financial inconsistencies.
- Stale state.
- Recurring incidents.
- Security boundary failures.
- Previously known failure patterns.
- Broken invariants.

It can flag and investigate, but production changes remain human-controlled unless an explicitly approved safe remediation workflow exists.

I. Database Verification / Reconciliation Capability

Responsible for direct database truth.

Checks:

- Stored values vs calculated values.
- Financial balances.
- Inventory quantities.

- Transaction history.
- Referential integrity.
- Orphaned records.
- RPC outcomes.
- State consistency.

Financial reconciliation is a first-class verification capability, not a UI test.

J. Browser / E2E Verification Capability

Technology baseline: Playwright.

Checks:

- Real browser behavior.
- User flows.
- UI state.
- Cross-screen synchronization.
- Navigation.
- Form submission.
- Image/data rendering.
- Loading/error/empty states.
- Permissions through actual UI paths.

Database correctness alone does not prove UI correctness.

K. API / Integration Verification Capability

Checks:

- Request/response behavior.
- RPC/API contracts.
- Authorization.
- Validation.
- Error responses.
- Idempotency.
- Retry behavior.
- Integration boundaries.

L. Security Verification Capability

Checks:

- RLS.
- Authorization boundaries.

- Role/permission enforcement.
- Direct URL/API/RPC bypass.
- Cross-tenant/cross-scope access.
- Authentication assumptions.
- Secret exposure.
- Security regression scenarios.

Security must be enforced at the server/data layer, not only by hiding UI controls.

M. Visual Regression Capability

Technology direction: Playwright screenshots initially.

Checks:

- Visual regressions.
- Component consistency.
- Missing images/placeholders.
- Layout breakage.
- Responsive states.

This is a later capability unless explicitly enabled earlier.

N. Performance Capability

Checks:

- Latency.
- Slow queries.
- Heavy screens.
- Load behavior.
- API/RPC timing.
- Client performance.

This is a later capability unless explicitly enabled earlier.

O. Memory Keeper

Owns engineering memory updates.

Records:

- Incident.
- Reproduction.
- Root cause.
- Evidence.
- Resolution.

- Regression test.
- Prevention rule.
- Related decision.
- Confidence/status.

A hypothesis must never be stored as a verified learning.

P. Human / Owner Gate

Human approval is mandatory for:

- Business-rule changes.
- Financial logic changes.
- Security architecture changes.
- Authentication changes.
- Major schema changes.
- Production data changes.
- Destructive operations.
- Infrastructure/billing decisions.
- Production deployment/release.
- Acceptance of known risk.
- Conflicting requirements.

4. Risk-Based Autonomy

Tier 1 — Safe / Routine / Autonomous

Examples:

- Read code.
- Inspect schema.
- Write tests.
- Run tests.
- Analyze test failures.
- Clean QA-only data.
- Fix routine bugs with regression coverage.
- Update documentation when no business rule changes.
- Collect evidence.

Tier 2 — Controlled / Propose → Human Approve

Examples:

- Business-rule changes.
- Financial logic.
- Authentication changes.
- Security architecture.
- Major schema migrations.
- Production data changes.
- Destructive operations.
- Infrastructure/billing changes.

Tier 3 — Forbidden without explicit human control

- Expose secrets.
- Bypass authorization.
- Weaken security to make tests pass.
- Silently replace business rules.
- Declare PASS/VERIFIED without evidence.
- Modify production data without approval.
- Deploy to production without human release approval.

5. Technology Baseline

Existing Agri Shop verified baseline

The architecture inventory recorded:

Layer	Current technology / mechanism
Client	React Native + Expo
Language	TypeScript
State	Zustand stores
Backend	Supabase
Database	PostgreSQL through Supabase
Authorization	RLS + server-side RPC enforcement
Database writes	RPC-oriented architecture; direct writes are an architecture violation for protected business operations
Auth	Farmer App account creation/login must support email or phone. Guest use remains allowed where appropriate. Authentication must be handled by a secure production auth system; historical mock-auth paths must never be treated as production auth
Storage	Supabase Storage for images
Unit/integration tests	Vitest
Browser/E2E	Playwright

Layer	Current technology / mechanism
CI/CD	GitHub Actions
Mobile artifact	Android APK build gate
QA environment	Isolated Supabase-hosted QA database
Telemetry	Observation Bot / useObservationBot
Business signals	Signal Engine
Evidence	Structured evidence test system
Engineering docs	AGENTS.md, LEARNINGS.md, DECISION_REGISTER.md, DEFINITION_OF_DONE.md, ACCEPTANCE_CRITERIA.md, ARCHITECTURE.md, AUTH_SPEC.md, THREAT_MODEL.md

Important historical lesson

A previous Agri Shop implementation changed from React Web to React Native Expo after development had already started, creating major rework. Therefore:

For the Farmer App, platform must be explicitly locked before feature implementation.

The Farmer App must not inherit the Agri Shop stack merely because it existed before.

Farmer App technology status

The following baseline decisions are **ACCEPTED** unless explicitly superseded:

- Mobile: React Native + Expo + TypeScript.
- Backend/data: Supabase + PostgreSQL.
- Server layer: Supabase Edge Functions + PostgreSQL RPCs where protected business operations require server-side integrity.
- Customer AI: OpenAI API / Responses API.
- CI/CD: GitHub + GitHub Actions.
- Authentication: email or phone signup/login; guest use may remain available for appropriate low-risk utility.
- Notifications: push-first; provider details remain implementation-configurable.
- Weather: provider abstraction; final provider requires commercial/quality validation.
- Market rates: provider/ingestion adapter; AMIS Punjab is the initial candidate where coverage exists.
- Crop diagnosis: multimodal AI + farmer/crop context + agricultural knowledge retrieval + confidence/safety gate + escalation.

The following remain **OPEN / IMPLEMENTATION DEPENDENCIES or EVIDENCE REQUIRED**, not reasons to repeatedly ask settled questions:

- Provider credentials/account configuration.
- Commercial/legal approval for external providers.
- Exact analytics vendor.
- Exact observability vendor.
- Final media-processing/transcoding implementation.

- Any worker-hosting choice required for future development automation.

The development-agent architecture itself remains **OPEN / FEASIBILITY RESEARCH REQUIRED** and must not be implemented or locked from assumption.

6. Target Engineering Intelligence Architecture

The target system should reuse the existing product engineering foundation rather than create a disconnected “AI intelligence” product.

Core storage/capabilities

1. **Business Rule Knowledge Base**
   - Machine-readable business rules.
   - Rule IDs.
   - Scope/entity.
   - Invariants.
   - Source.
   - Status.
   - Version.
   - Related RPC/API/UI.
   - Conflict detection.
2. **Evidence Store**
   - Expected.
   - Actual.
   - Evidence source.
   - Environment.
   - Timestamp.
   - Test/run ID.
   - Result.
   - Related requirement/change/incident.
3. **Incident Store**
   - Symptom.
   - User action.
   - File/function/entity.
   - Timestamp.
   - Error evidence.
   - Expected/actual.

- Root cause.
- Resolution.
- Verification.
- Prevention.
- Classification.

4. **Engineering Memory**
   - Verified learnings.
   - Failure patterns.
   - Root causes.
   - Fixes.
   - Regression tests.
   - Prevention rules.
   - Links to incidents.
5. **Business-Rule / Dependency Graph**
   - Entity relationships.
   - Data dependencies.
   - RPC/function dependencies.
   - RLS/security dependencies.
   - UI dependencies.
   - Business invariants.
   - Change impact analysis.
6. **Financial Reconciliation**
   - Canonical calculations.
   - Stored-vs-derived comparisons.
   - Customer balance reconciliation.
   - Credit reconciliation.
   - Inventory reconciliation.
   - Purchase/payable reconciliation.
   - Profit/COGS reconciliation where applicable.
7. **Regression Suite Manager**
   - Maps changes to affected tests.
   - Identifies regression scope.
   - Runs required suites.
   - Records results and evidence.

7. Observation / Incident Model

The existing Observation Bot should be extended rather than replaced.

Current observation capability:

- User actions.
- Errors.
- Performance.
- Screen tracking.
- Local persistence.
- Human-readable reports.

Target:

Runtime → Observation → Incident record → Context collection → Pattern matching → Investigation → Verified resolution → Regression test → Learning/prevention rule

The system must not:

- Automatically weaken security.
- Treat similarity as proof.
- Auto-fix production issues without approved authority.
- Override business judgment.

8. Verification Evidence Rules

Bug fix

Required:

- Reproduction.
- Root-cause evidence.
- Fix.
- Regression test.
- Relevant CI/test pass.
- Browser/DB evidence where applicable.

New feature

Required:

- Unit/integration tests as applicable.
- E2E for meaningful user journeys.
- Manual/design verification where automated verification cannot establish UX correctness.
- Evidence record.

Schema change

Required:

- Migration.
- Integrity checks.
- Rollback/recovery plan.
- Regression impact analysis.

Security change

Required:

- Attack/bypass scenario.
- Defense verification.
- Regression coverage.

Performance change

Required:

- Before/after metrics.
- Relevant load/timing evidence.

9. Verification Chain

Requirement → Business Rule → Architecture → Backend → Database → Authorization → Frontend → UI State → Cross-screen → E2E → Regression → Release

Known limitations from the previous architecture:

- Requirement → business rule: requires formal rule store.
- Business rule → architecture: human/technical judgment may be required.
- Backend → database: high coverage through RPC/reconciliation.
- Authorization/RLS: high coverage when direct boundary tests exist.
- Frontend/UI: cannot be proven by database tests alone.
- Visual state: requires browser/screenshot verification.
- Full release: requires human approval.

10. Business Rule and Dependency Graph

Every important entity should be represented as:

Entity → Rules → Dependencies → Read paths → Write paths → RPC/API → Permissions → UI surfaces → Tests → Evidence

Example:

Customer → Sales → Payments → Credit → Reminders → Outstanding → Notifications

Example:

Product → Variant → Inventory → Purchase → Sale → Return → Void → COGS

Before changing an entity, the Orchestrator must perform impact analysis against this graph when available.

11. State-Machine Rule

Any meaningful lifecycle must define:

- States.
- Allowed transitions.
- Invalid transitions.
- Actor permissions.
- Side effects.
- Retry/idempotency.
- Audit record.
- Terminal states.

Server-side enforcement is required for critical transitions.

12. Persistent Documentation Contract

The engineering source of truth is composed of focused documents rather than one giant prompt.

Required operating files:

- **ROLE.md** — agent roles, responsibilities, authority.
- **PRODUCT_RULES.md** — product/business rules.
- **DESIGN_THINKING.md** — design reasoning framework.
- **DECISIONS.md** — accepted/proposed/superseded decisions.
- **ARCHITECTURE.md** — technical architecture and dependencies.
- **IMPLEMENTATION_RULES.md** — coding/implementation constraints.
- **QA_RULES.md** — verification/evidence/regression rules.
- **LEARNINGS.md** — verified failure memory.
- **AGENTS.md** — high-level operating manual and non-negotiable rules.

These files should remain synchronized with the Notion source of truth.

13. Non-Negotiable Rules

1. Do not invent business rules.
2. Do not silently change accepted decisions.
3. Do not treat proposals as requirements.
4. Do not claim VERIFIED without evidence.
5. Do not let UI-only checks substitute for database/security verification.
6. Do not let database checks substitute for UI verification.

7. Do not perform destructive production QA by default.
8. Do not expose secrets.
9. Do not bypass authorization.
10. Do not weaken security to make tests pass.
11. Do not hard-delete financial/history records where the domain requires preservation.
12. Do not silently change historical financial values.
13. Do not create duplicate source-of-truth systems.
14. Do not implement a new feature without checking existing decisions and dependencies.
15. Do not repeatedly ask for information already present in the project knowledge base.
16. When required information is missing, ask a targeted question rather than guessing.
17. When a decision changes, preserve the previous decision and document the reason/evidence for the change.
18. Human approval remains mandatory for business, financial, security, production, and irreversible decisions.

14. Failure-Prevention Rules Learned from Agri Shop

The previous Agri Shop architecture review identified these recurring failure modes:

- Platform decision changed too late.
- Authentication was mocked instead of implemented securely.
- Planned RPC architecture was bypassed by direct client writes.
- Financial operations were split instead of atomic.
- FIFO/financial logic was deferred.
- Business rules were scattered into UI code.
- Data access became monolithic instead of domain-separated.
- Shared hooks/components were not created early.
- Agent infrastructure consumed scope intended for the core product.
- Planning documents were not continuously validated against working code.
- Lack of version control/rollback created unnecessary risk.

Therefore for the Farmer App:

Architecture decisions must be locked before implementation.

Core authentication and data integrity must be proven before feature expansion.

The AI development system itself is not a customer-facing product feature.

Every architecture claim must be tested against real code/runtime evidence within the development cycle.

15. Release Gate

A release candidate must have:

- Requirement understood.
- Accepted decision confirmed.
- Impact analysis completed where relevant.
- Implementation complete.
- Required unit/integration tests pass.
- Required E2E tests pass.
- Database reconciliation pass where relevant.
- Security verification pass where relevant.
- Visual/UI verification where relevant.
- No unresolved critical regression.
- Evidence stored.
- Human release approval.

Rollback/recovery must exist before production risk is accepted.

16. Source Documents Reconstructed Into This Operating System

The architecture was reconstructed from the project's Library documents:

- **Pasted markdown(2).md** — CTO project audit, architecture lessons, platform/auth/RPC/FIFO/transaction/agent-system failures, and future engineering standards.
- **Pasted markdown(8).md** — multi-layer QA architecture, verification coverage, Lead QA Mike vs Mike Assistant, browser automation gaps, safety layers, and structural limitations.
- **Pasted markdown(9).md** — concrete automation workflows, verification examples, regression/evidence model, autonomy expectations, and human-controlled boundaries.
- **Pasted markdown(20260913-111020).md** — current Agri Shop architecture inventory, Observation Bot assessment, engineering memory, incident architecture, business-rule graph, verification automation, and Phase 4 roadmap.
- **Pasted markdown(20260913-103457).md** — CI/auth/QA architecture evidence and verified test/build baseline from that phase.
- **Agri_Shop_Management_PRD_Final.pdf** — finalized Agri Shop business requirements and locked MVP rules.

The original documents remain historical source material. This page is the normalized operating baseline for future AI work.

17. Farmer App-Specific Open Decisions — resolved at architecture level

The user has delegated technical/provider research to the architecture role. Do not repeatedly ask settled questions; escalate only credentials, legal/commercial approval, or genuinely new product decisions. The development-agent architecture is intentionally **NOT DECIDED** and must be researched for feasibility before any implementation decision.

Accepted baseline

- Mobile: React Native + Expo + TypeScript for Android and iOS.
- Backend/data: Supabase + PostgreSQL.
- Server layer: Supabase Edge Functions + PostgreSQL RPCs.
- Customer AI: OpenAI API / Responses API.
- Development-agent architecture: **OPEN / FEASIBILITY RESEARCH REQUIRED**. First investigate whether the desired assigned-agent model can actually accept multiple tasks, release workers independently, run concurrent work safely, checkpoint/resume, isolate workspaces, and hand evidence between roles. Only after that research should an architecture be proposed.
- Development-agent system: **OPEN / RESEARCH REQUIRED**. Do not implement or lock an agent architecture until feasibility, concurrency, task handoff, execution isolation, cost, speed, failure/retry behavior, and actual tool/runtime limitations have been verified.
- Auth: **email or phone signup/login**. Anonymous browsing/use may remain available where appropriate, but account creation must support both email and phone.
- Notifications: push-first; Expo Notifications / Expo Push Service for MVP.
- CI/CD: GitHub + GitHub Actions.
- Market data: AMIS Punjab where coverage exists, behind an ingestion/provider adapter.
- **Weather: provider abstraction. WeatherAPI.com Free is the initial MVP candidate** because its current free plan provides 100K calls/month, current weather and 3-day forecasts, and its terms permit commercial use with attribution. Keep the provider behind an adapter; upgrade/change provider only if scale, feature or reliability needs require it. Open-Meteo remains a free non-commercial evaluation/prototyping option. cite turn215206search1 turn215206search10 turn215206search2
- Market: AMIS Punjab behind a permitted ingestion/provider adapter; machine-readable access and commercial/legal permission must be confirmed before production ingestion. cite turn545971search4
- Voice: text language scope remains Urdu + Punjabi + Saraiki + Sindhi + Pashto; voice is capability/quality gated per language/provider. Saraiki voice is not covered in the current Google Speech-to-Text language list reviewed. cite turn306136search0
- Crop diagnosis: multimodal AI + farm/crop context + knowledge retrieval + confidence/safety gate + escalation.

Remaining implementation dependencies

- Provider credentials and account configuration.
- Commercial/legal approval for external data sources.
- Exact analytics vendor.
- Exact worker-hosting vendor if GitHub Actions is insufficient for long-running coding work.

These are implementation dependencies, not reasons to stop architecture.

17A. Previous Open-Decision Questions (historical)

These are the only technology/architecture questions the agent should escalate before locking the Farmer App engineering foundation:

1. Is the Farmer App also **React Native + Expo + TypeScript**?
2. Is **Supabase/PostgreSQL** the intended backend/data foundation for the Farmer App?
3. What AI orchestration/runtime should be used for the development agent system, and where should its persistent memory/evidence store live?
4. Which AI/vision provider is intended for crop diagnosis?
5. What authentication model is required for farmers?
6. Which notification channels are required first?
7. Which external providers will be used for weather and market rates?
8. What is the deployment/CI environment?

Until these are answered, the agent should inherit the **engineering rules and QA architecture**, but not falsely lock the Farmer App's technology stack.

18. Operating Command for Future AI Agents

When the user gives a new requirement, the agent should behave as follows:

Load context → classify requirement → check decisions → check design → check architecture → inspect dependencies → perform second-order impact analysis → identify missing information → propose smallest coherent implementation → ask only high-value unresolved questions → implement only within authority → verify independently → record evidence → update memory/decision state → report exactly what is verified and what remains open.

The agent should not ask the user to repeat information that is already stored here or in the linked product source of truth.